COURSE TITLE	**EXPLOIT DEVELOPMENT AND SHELL SCRIPTING**
COURSE CODE	**01CC0606**
COURSE CREDITS	**3**

Objective:

1 Students will be able to apply concepts of program execution, memory layout, and vulnerability exploitation to develop proof-of-concept exploits and shell scripts for security tasks, analyze software vulnerabilities including buffer overflows, heap issues, and format-string flaws using debugging and fuzzing techniques, and evaluate exploitation strategies along with modern mitigation mechanisms to understand their impact on exploit development and post-exploitation processes

Course Outcomes: After completion of this course, student will be able to:

1 Apply debugging tools and techniques to identify, analyze, and understand software vulnerabilities in different programs

2 Implement shell scripting techniques to automate system administration and security-related tasks effectively.

3 Analyze various types of vulnerabilities and exploit techniques to understand their behavior and impact on systems

4 Evaluate exploitation strategies and modern security mitigation techniques to determine their effectiveness.

5 Design and develop custom exploits and automated scripts for penetration testing and security operations.

Pre-requisite of course: Solid understanding of C programming language, including pointers and memory management. Familiarity with a compiled language (like C) and basic compilation process. Basic understanding of computer architecture, including CPU registers and memory. Familiarity with a command-line interface (Linux is preferred). Basic understanding of networking concepts.

Teaching and Examination Scheme

Theory Hours	Tutorial Hours	Practical Hours	ESE	IA	CSE	Viva	Term Work
2	0	2	50	30	20	25	25

DR. MUNINDRA HASMUKHBHAI LUNAGARIA
Digitally signed by (Name of HOD)

DR. RAJENDRASINH BAHADURSINH JADEJA
Digitally signed by (Name of Dean/ Principal)

Contents : Unit	Topics	Contact Hours
1	**Foundations of Program Execution and Vulnerabilities** Introduction to Exploit Development: Concepts, goals, ethical considerations, Computer Architecture Review (relevant to exploitation): CPU registers, memory organization (stack, heap, data, text segments), Program Compilation and Execution Process, Introduction to Assembly Language (basic x86/x64):, Understanding function calls, stack operations, Debugging Fundamentals: Using tools like GDB or WinDbg to inspect memory and registers,, Introduction to Common Vulnerability Classes: Buffer overflows, integer overflows, format string vulnerabilities, race conditions (overview)	7
2	**Stack Buffer Overflow Exploitation** Deep Dive into Stack Buffer Overflows: How they occur, overwriting return addresses, Controlling Program Execution Flow, Introduction to Shellcode: Purpose, writing basic shellcode (e.g., execve("/bin/sh"), Shellcode Encoding and Decoding Techniques (e.g., XOR, shikata ga nai - basic concepts), Developing Stack Buffer Overflow Exploits (non-networked examples), Basic Protections and Bypass (e.g., No-Execute bit/DEP - conceptual)	6
3	**Introduction to Shell Scripting for Security** Shell Scripting Fundamentals (Bash): Basic syntax, variables, input/output, File Operations and Permissions in Scripts, Conditional Statements and Loops, Command Substitution and Piping, Essential Shell Commands for Security Tasks: grep, awk, sed, find, User and Permission Management using Scripts	6
4	**Advanced Vulnerability Classes and Exploitation Concepts** Heap Overflows: Understanding heap memory management and common heap exploitation techniques (introduction), Format String Vulnerabilities: How they occur, reading/writing memory using format strings, Return-Oriented Programming (ROP): Concepts and basic chain building (introduction), Integer Overflows and Underflows: Detecting and exploiting integer vulnerabilities, Introduction to Fuzzing for vulnerability discovery (basic concepts)	6
5	**Integrating Shell Scripting and Exploit Development** Using Shell Scripts for Reconnaissance: Automating port scanning (nmap scripting basics), service banner grabbing, Writing Scripts for Vulnerability Scanning and Analysis (integrating command-line tools), Automating Exploit Delivery and Execution using Shell Scripts, Creating Simple Security Tools using Shell Scripting, Post-Exploitation, Scripting: Automating data exfiltration or privilege escalation tasks (in a controlled lab), Defense and Mitigation: Introduction to exploit mitigation techniques and how they affect exploitation	7
Total Hours		**32**

DR. MUNINDRA HASMUKHBHAI LUNAGARIA
Digitally signed by (Name of HOD)

DR. RAJENDRASINH BAHADURSINH JADEJA
Digitally signed by (Name of Dean/ Principal)

Suggested List of Experiments:

Contents : Unit	Topics	Contact Hours
1	**Practical 1** Setting up a Safe Exploit Development Lab: Installing and configuring virtual machines (e.g., using VirtualBox or VMware) and vulnerable application environments (e.g., Metasploitable, purposefully vulnerable VMs)	2
2	**Practical 2** Basic Shell Scripting Fundamentals: Writing and executing simple Bash scripts for file manipulation, variable usage, and command execution.	2
3	**Practical 3** Writing Shell Scripts for Basic System Information Gathering: Creating scripts to extract and format system information (OS version, running processes, network configuration)	2
4	**Practical 4** Debugging a Simple C Program: Using GDB to set breakpoints, step through code, inspect variables, and examine memory layout.	2
5	**Practical 5** Identifying a Stack Buffer Overflow: Analyzing a provided vulnerable C program and using a debugger to confirm a buffer overflow condition	2
6	**Practical 6** Developing a Basic Stack Buffer Overflow Exploit (Return Address Overwrite): Writing a Python or C program to exploit the vulnerable application by overwriting the return address to redirect execution to a specific location within the program.	2
7	**Practical 7** Creating and Executing Simple Shellcode: Writing and testing basic shellcode (e.g., /bin/sh execution) in a controlled environment and embedding it in a simple program.	2
8	**Practical 8** Using Shell Scripting for Automated Port Scanning: Writing a Bash script that utilizes nmap or other tools to scan a range of IP addresses for open ports and report the results	2
9	**Practical 9** Automating Exploit Execution with a Shell Script: Creating a shell script that compiles or runs a proof-of-concept exploit program and handles its input/output (in a controlled environment).	2
10	**Practical 10** Analyzing a Simple Exploit Script: Examining a provided script (e.g., a Python script for a simple web vulnerability or a Bash script for a known OS vulnerability) to understand its functionality and how it leverages scripting for exploitation or automation.	2
Total Hours		**20**

DR. MUNINDRA HASMUKHBHAI LUNAGARIA
Digitally signed by (Name of HOD)

DR. RAJENDRASINH BAHADURSINH JADEJA
Digitally signed by (Name of Dean/ Principal)

Textbook :

1 “Hacking: The Art of Exploitation, Jon Erickson, No Starch Press, 2008

References:

1 Practical Binary Analysis: Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly, Practical Binary Analysis: Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly, Dennis Andriesse , No Starch Press, 2018

2 Shellcoder’s Handbook: Discovering and Exploiting Security Holes, Shellcoder’s Handbook: Discovering and Exploiting Security Holes, Chris Anley et al. , Wiley Publishing, Inc., 2007

Suggested Theory Distribution:

The suggested theory distribution as per Bloom’s taxonomy is as follows. This distribution serves as guidelines for teachers and students to achieve effective teaching-learning process

Distribution of Theory for course delivery					
Remember / Knowledge	**Understand**	**Apply**	**Analyze**	**Evaluate**	**Higher order Thinking / Creative**
0.00	4.00	28.00	28.00	20.00	20.00

Instructional Method:

1 The course delivery method will depend upon the requirement of content and need of students. The teacher in addition to conventional teaching method by black board, may also use any of tools such as demonstration, role play, Quiz, brainstorming, MOOCs etc

2 The internal evaluation will be done on the basis of continuous evaluation of students in the laboratory and class-room

3 Practical examination will be conducted at the end of semester for evaluation of performance of students in laboratory.

4 Students will use supplementary resources such as online videos, NPTEL videos, e-courses, Virtual Laboratory

Supplementary Resources:

1 NPTEL Course: Foundations of Cyber Security (Exploitation & SE) – IIT Kanpur (Prof. Manindra Agrawal)

2 “Software Security” – University of Maryland (Coursera)

3 Virtual lab / VM : Metasploitable 2 & 3